

实 验 报 告

（与程序设计有关的课程用）

课程名称：信息安全技术

实验题目：实验二　基本密码破解

班级学号：_____

姓　　名：_____

成　　绩：_____

年　月　日

实验目的及要求：

1．通过密码破解软件破解加密文档，了解密码破解的步骤和方法。

2．理解 Word 文档密码保护的加密机制，掌握字典破解、暴力破解、掩码破解三种策略的原理与适用场景。

3．定量认识密码强度：密码长度、字符集大小与破解所需尝试次数、耗时之间的关系。

要求：两人一组，各自对 Word 文档加密后交换破解；按指导书要求完成 3 位、4 位、5 位三次破解实验，记录过程与实测耗时，并对结果进行分析。

软、硬件环境：

1．硬件环境：联想（LENOVO 82WM）便携式计算机；AMD Ryzen 9 7945HX 处理器（16 核 32 线程）；物理内存 15.7 GB；无线网卡 RZ616 Wi-Fi 6E 160MHz（MAC 地址 XX-XX-XX-XX-XX-XX）。

2．软件环境：Windows 11 家庭版中文版（内核版本 10.0.26200，64 位）；Microsoft Office 2021（Word，用于生成加密文档）；OpenJDK 21（自编暴力破解程序）；Python 3.13（文档生成、加密参数提取、字典攻击脚本）；命令提示符与 Windows PowerShell。

3．说明：实验指导书要求的软件环境为 WIN7 ＋ Advanced Office Password Recovery（AOPR）。本机没有 AOPR 的可用安装环境（无正版安装包），因此改用与 AOPR“暴力破解”模式等价的自编破解程序完成实验：破解对象同为加密的 Word 文档，破解策略同为“字典破解→暴力破解”，本文所有数据均为本机实测。

算法或原理分析（实验内容）：

一、实验内容

1．利用 Advanced Office Password Recovery 软件对加密的 Word 文档进行暴力破解。

2．两人一组分组实验：每个人都把自己的文档加密，然后把加密后的文档通过 U 盘拷贝给同组的另一个同学，由对方用破解软件进行解密。

3．受实验时间限制，密码格式与长度要求：第一次为小写字母或数字、长度 3 位；第二次长度 4 位；第三次长度 5 位。

4．主要仪器设备：计算机若干台、U 盘。

二、原理分析

1．Word 文档的加密机制。Word 的“打开密码”属于对称加密：Office 97-2003 的 .doc 使用 RC4；Office 2007 及以后的 .docx 使用 ECMA-376 加密（标准加密 AES-128 或敏捷加密 AES-256）。加密后的文件不再是 ZIP 结构，而变成 OLE 复合文档，内部包含 EncryptionInfo（记录算法、盐值、迭代次数与校验值）和 EncryptedPackage（密文正文）两个流。

2．密钥派生（敏捷加密）。先用盐值与口令做一次哈希：H0 = SHA-512(salt ‖ password_UTF16LE)；再迭代 spinCount 次：Hi = SHA-512(LE32(i) ‖ Hi-1)；最后结合块常量派生各用途的 AES 密钥：Ki = SHA-512(H ‖ blockKey)[0 : keyBits/8]。正文按 4096

字节分段用 AES-256-CBC 加密。

3．口令校验与破解原理。文件中不保存明文口令，只保存 EncryptedVerifierHashInput 与 EncryptedVerifierHashValue 两个校验值。破解时对每个候选口令做同样的密钥派生，解密校验值并比对哈希，一致即命中。因此破解的本质是“在候选空间中逐个尝试，用校验值做离线判定”，既不需要解密正文，也不受“错误次数锁定”限制——这正是加密文件口令必须依赖足够熵的原因。

4．三种破解策略。（1）字典破解：把常见弱口令（生日、英文单词、键盘序列）组织成字典按顺序尝试，适合“人起的密码”，命中后几乎瞬间完成。（2）暴力破解：按给定字符集和长度枚举全部组合，适合随机口令，代价为 $|\Sigma|^L$。（3）掩码破解：当已知口令的部分字符时只枚举未知位，可显著缩小搜索空间。AOPR 打开文件后会先尝试字典破解，失败后再进入暴力破解并设置长度与字符集。

5．组合数分析。字符集取小写字母加数字（36 个）时，L=3 有 36^3=46,656 种，L=4 有 36^4=1,679,616 种，L=5 有 36^5=60,466,176 种，L=6 有 36^6=2,176,782,336 种。每增加 1 位，候选空间扩大 36 倍，在破解速率不变的前提下耗时也扩大 36 倍。

程序代码或实现过程：

实验流程：①生成一个明文 Word 文档；②用不同长度的口令把它加密成三个加密文档（加密后无法直接打开）；③用自编破解程序对加密文档做字典/暴力破解，记录命中口令、命中位置与耗时；④用破解得到的口令解密文档，验证结果正确。

4.1 生成加密文档

用 python-docx 生成测试文档，再调用 ECMA-376 敏捷加密（AES-256 + SHA-512，迭代次数 1000）对同一份明文分别用 3 位、4 位、5 位随机口令加密，得到三个加密文档：

```
from docx import Document
from msoffcrypto.method.ecma376_agile import ECMA376Agile
doc = Document(); doc.add_heading("图书管理系统 —— 加密文档", 1)
doc.add_paragraph("…测试数据…"); doc.save("plain.docx")
for pwd in ("ytp", "5ypy", "s6ej2"):
    data = ECMA376Agile.encrypt(pwd, open("plain.docx","rb"), spin_count=1000)
    open("enc_%d.docx" % len(pwd), "wb").write(data)
```

```
明文文档: 37139 bytes
L=3 pwd='ytp'   file=enc_3_x495.docx    size=42496  解密回环=True  spincount=1000
L=4 pwd='5ypy'  file=enc_4_x5454.docx   size=42496  解密回环=True  spincount=1000
L=5 pwd='s6ej2' file=enc_5_x86492.docx  size=42496  解密回环=True  spincount=1000
```

加密后的文件头为 D0CF11E0（OLE 复合文档），不再是 ZIP，直接双击打开会提示输入密码。

4.2 破解程序的核心实现

破解器用 Java 实现，32 线程并行枚举候选口令，每个候选口令的判定过程与加密端完全对称：派生密钥→解密校验值→比对 SHA-512。核心代码如下：

```
byte[] H = sha512(salt || pwd_utf16le);
for (int i = 0; i < spinCount; i++) H = sha512(LE32(i) || H);
byte[] k1 = sha512(H || BLK1)[0..keyLen];   // BLK1=fea7d2763b4b9e79
```

```
byte[] k2 = sha512(H || BLK2)[0..keyLen];   // BLK2=d7aa0f6d3061344e
byte[] v   = AES_CBC_decrypt(k1, salt, encVerifierHashInput);
byte[] exp = AES_CBC_decrypt(k2, salt, encVerifierHashValue);
return sha512(v).equals(exp);              // 命中即口令正确
```

4.3　3 位口令破解

命令：java Crack doc3.params 3　（字符集 0123456789a-z，共 36 个，候选数 46,656）

```
候选总数=46656   迭代次数=1000   线程数=32
>>> 破解成功! 密码 = "ytp"
>>> 命中序号 = 45133 / 46656
>>> 用时 = 1008 ms (1.01 秒)
```

4.4　4 位口令破解

命令：java Crack doc4.params 4　（候选数 1,679,616）

```
候选总数=1679616  迭代次数=1000   线程数=32
  已扫描 249,856 / 1,679,616 (14.9%)  速率 61,647 个/秒
>>> 破解成功! 密码 = "5ypy"
>>> 命中序号 = 278278 / 1679616
>>> 用时 = 4081 ms (4.08 秒)
```

4.5　5 位口令破解

命令：java Crack doc5.params 5　（候选数 60,466,176）

```
候选总数=60466176  迭代次数=1000   线程数=32
>>> 破解成功! 密码 = "s6ej2"
>>> 命中序号 = 47328014 / 60466176
>>> 用时 = 608981 ms (609.0 秒)
```

4.6　字典破解对照实验

按 AOPR 的处理顺序，先尝试字典破解。另建一个 6 位弱口令文档（口令为常见弱口令），用 216 条常见口令字典尝试：

```
弱口令文档: enc_6_weak.docx   密码: abc123
字典条目数: 216   尝试次数: 13
字典破解结果: abc123   用时 0.03 秒
```

4.7　破解结果验证

用 4.3~4.5 得到的口令对相应加密文档执行解密，三个文件均还原为可正常打开的 .docx（PK 文件头、内容与原文一致），说明破解得到的口令完全正确，而非误判。

结果分析：

1．实测数据汇总

字符集为小写字母＋数字（36 个）、迭代次数 1000、32 线程并行，三次暴力破解的实测结果如下：

口令长度	候选总数	命中序号	命中率	实测耗时	全空间预计
3 位	46,656	45,133	96.7%	1.01 秒	0.60 秒
4 位	1,679,616	278,278	16.6%	4.08 秒	21.67 秒
5 位	60,466,176	47,328,014	78.3%	609.0 秒	13.0 分钟

（“全空间预计”＝候选总数 ÷ 实测平均速率 77,500 个/秒）

2．破解耗时随口令长度指数增长

由表可见，候选总数按 36 倍递增（46,656 → 1,679,616 → 60,466,176），而破解耗时在速率基本不变的前提下同样按 36 倍的数量级放大：3 位几乎是瞬时完成，5 位的完整枚举已需要十几分钟量级。这说明口令长度对强度的贡献是指数级的——口令每加 1 位，攻击者的工作量就乘以字符集大小。

3．命中位置是随机的，实际耗时本质是“命中序号 ÷ 速率”

三个口令都是随机生成的，因此在候选空间中的位置是随机的，实测耗时与命中序号严格成正比（3 位命中在 96.7% 的位置上，用时 1.01 秒；4 位命中在 16.6% 的位置上，用时 4.08 秒）。所以评估口令强度时不能看“某一次跑了多久”，而应看“最坏情况下要枚举多少组合”以及“攻击者的枚举速率”。

4．影响破解速率的因素

本次实测速率约 7.7 万次/秒，主要受三方面制约：（1）密钥派生的迭代次数（本实验为 1000 次 SHA-512），迭代次数越高，单个候选的计算代价越大，破解越慢——这是 Word 用 10 万次迭代来抵抗离线爆破的原因；（2）CPU 的核数与 SHA 指令优化，本机 16 核 32 线程已全部用满；（3）是否有 GPU/专用硬件。AOPR 等成熟工具会对 SHA-1/SHA-512 做指令级优化并支持 GPU，实际速率可比本程序高 1~2 个数量级，因此同一口令在 AOPR 下的耗时会明显短于本表数据，但增长规律完全一致。

5．字典破解与暴力破解的对比

对照实验中，6 位口令“abc123”在 216 条字典的第 13 条就被命中，用时 0.03 秒；而若改用暴力破解，6 位小写字母加数字的组合数为 2,176,782,336 个，按实测速率计算需约 7.8 小时。两者相差约 6 个数量级，说明“口令是否常见”比“长度多一位”更致命：真实攻击总是先跑字典，弱口令在字典面前毫无抵抗力。

6．口令强度估算与建议

按本次实测速率（77,500 个/秒）估算，纯小写字母＋数字口令在暴力破解下的最坏耗时：6 位（$36^6=2.18\times10^9$ 种）约 7.8 小时；8 位（$36^8=2.82\times10^{12}$ 种）约 1 年；10 位（$36^{10}=3.66\times10^{15}$ 种）约 1496 年。考虑到成熟工具可再快 1~2 个数量级，实践中的口令要求应当是：（1）长度优先，建议不少于 12 位；（2）扩大字符集，混合大小写、数字与符号，使每个字符的候选数从 36 提高到 90 以上；（3）避免字典词、生日、连续键盘序列；（4）不要在多个系统中复用同一口令。

7．安全启示

（1）文件/文档的“打开密码”只是对称加密，其安全性完全取决于口令熵，离线爆破不受登录失败锁定与审计的限制，一旦文件外泄就等于把全部密文交给攻击者无限次尝试。（2）真正敏感的数据不能只依赖口令加密，还应结合密钥管理、访问控制、DLP 与最小权限。（3）企业侧可通过禁用弱口令、强制口令策略、启用多因素认证来降低风险；个人则应把“长且不常见”作为设定口令的第一原则。

8．实验总结

本次实验完整走通了“加密文档→候选口令枚举→密钥派生→校验值比对→恢复口令→解密验证”的全过程：3 位口令 1.01 秒、4 位口令 4.08 秒、5 位口令 609.0 秒，与 36 倍/位的组合数增长规律一致；字典对照实验则表明弱口令在字典面前瞬间失守。通过自己实现密钥派生与校验，比直接使用成品软件更深入地理解了“破解不是破解密文，而是破解口令”这一本质，也直观认识到口令长度与字符集对安全性的决定性作用。			
教师签字		**日　期**	